

TezKart – Full Frontend Project Documentation

Quick Commerce E-Commerce Frontend Application

Inspired By:

- [Zepto](#)
 - [Blinkit](#)
-

Table of Contents

1. Project Introduction
2. Project Objective
3. Project Description
4. Business Problem
5. Target Users
6. Core Features
7. Technology Stack
8. System Architecture
9. Frontend Architecture
10. Folder Structure
11. UI/UX Design System
12. Application Pages
13. Components Documentation
14. Routing Architecture
15. State Management
16. API Integration Layer
17. Authentication System
18. Cart Management System

19.	Search Functionality
20.	Responsive Design
21.	Performance Optimization
22.	Animations & User Experience
23.	Error Handling
24.	Testing Strategy
25.	Git & GitHub Workflow
26.	Build & Deployment
27.	Future Enhancements
28.	Project Workflow
29.	Resume Description
30.	Conclusion

1. Project Introduction

Project Name

TezKart

TezKart is a modern quick-commerce frontend application developed to provide a fast, responsive, and user-friendly online grocery shopping experience.

The project is inspired by modern grocery delivery platforms such as Zepto and Blinkit.

The frontend application focuses on:

- Speed
- Responsive UI
- Better shopping experience
- Real-time cart interaction
- Clean user interface
- Mobile-first design

2. Project Objective

The objective of this project is to build a professional e-commerce frontend application using modern frontend technologies and industry-level development practices.

The project helps users:

- Browse products
- Search groceries
- Add/remove cart items
- View offers
- Checkout products
- Experience responsive shopping UI

3. Project Description

TezKart is a frontend-only React application designed for online grocery ordering and quick commerce services.

The application includes:

- Product listing pages
- Category browsing
- Search system
- Product detail pages
- Cart functionality
- Authentication UI
- Checkout interface
- Responsive layouts

The project is designed using reusable React components and scalable architecture.

4. Business Problem

Problems Solved

Problem	Solution
Slow grocery ordering	Quick UI navigation
Poor mobile experience	Responsive mobile-first design
Complicated cart systems	Easy cart management
Unorganized product browsing	Category-based filtering
Slow checkout flow	Simplified checkout UI

5. Target Users

User Type	Description
Grocery Customers	Buy daily essentials
Mobile Users	Shop using smartphones
Students	Quick grocery ordering
Working Professionals	Fast shopping experience
Urban Customers	Instant delivery services

6. Core Features

Feature	Description
Responsive Navbar	Mobile navigation
Hero Banner	Promotional section
Product Listing	Grocery products
Search System	Instant searching

Feature	Description
Categories	Product grouping
Product Details	Detailed information
Add To Cart	Cart functionality
Cart Drawer	Side cart panel
Quantity Controls	Increase/decrease quantity
Checkout UI	Order placement flow
Authentication UI	Login/Register
Responsive Layout	Mobile/tablet/desktop
Toast Notifications	User feedback
Animations	Smooth transitions

7. Technology Stack

Layer	Technology
Frontend	ReactJS
Language	TypeScript
Styling	Tailwind CSS
UI Framework	Bootstrap
Routing	React Router DOM
State Management	Redux Toolkit
API Integration	Axios
Animations	Framer Motion
Notifications	React Toastify

Layer	Technology
Build Tool	Vite
Version Control	Git & GitHub

8. System Architecture

High-Level Architecture

Browser



React Frontend



Redux Store



Axios API Layer



Backend APIs

9. Frontend Architecture

Frontend Flow

UI Components



Pages



Redux Store



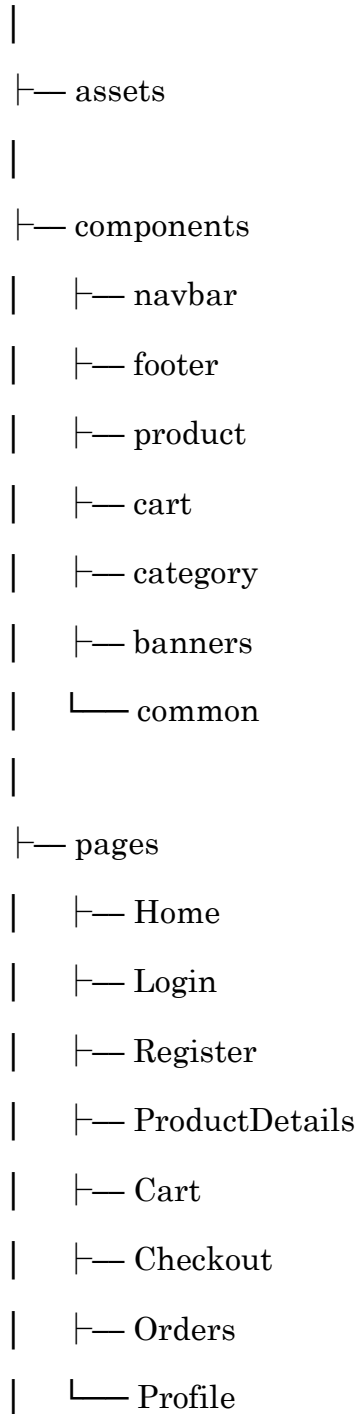
API Services



Backend

10. Folder Structure

src



```
|
|└─ redux
|  |└─ slices
|  └─ store.ts
|
|└─ services
|└─ routes
|└─ hooks
|└─ utils
|└─ layouts
└─ App.tsx
```

11. UI/UX Design System

Color Palette

Purpose	Color
Primary	Purple
Secondary	Green
Background	White
Error	Red
Success	Green

Typography

Element	Style
Heading	Bold

Element	Style
Product Title	Medium
Price	Semi-bold
Button Text	Bold

UI Principles

Principle	Description
Mobile First	Responsive design
Clean Layout	Better readability
Smooth Navigation	Better UX
Fast Interaction	Quick user actions

12. Application Pages

Page	Purpose
Home Page	Product listing
Login Page	User authentication
Register Page	User signup
Product Details	Product information
Cart Page	Shopping cart
Checkout Page	Order summary
Orders Page	User orders
Profile Page	User profile

13. Components Documentation

Main Components

Component	Purpose
Navbar	Navigation
ProductCard	Product display
HeroBanner	Promotions
SearchBar	Product searching
CartDrawer	Cart sidebar
Footer	Footer links

ProductCard Features

Feature	Description
Product Image	Product preview
Product Price	Pricing
Discount Badge	Offers
Add Button	Cart functionality

14. Routing Architecture

Route	Page
/	Home
/login	Login
/register	Register
/product/:id	Product details
/cart	Cart

Route	Page
/checkout	Checkout

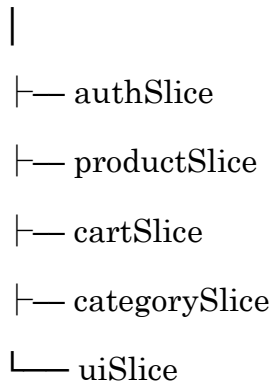
React Router Features

Feature	Purpose
Dynamic Routes	Product details
Nested Routes	Layout structure
Protected Routes	User security

15. State Management

Redux Store Structure

store



Slice Responsibilities

Slice	Purpose
authSlice	User authentication
productSlice	Product data
cartSlice	Cart operations

Slice	Purpose
categorySlice	Slice Categories
uiSlice	Theme/loading

16. API Integration Layer

Axios Responsibilities

Feature	Purpose
API Calls	Backend communication
JWT Handling	Token management
Error Handling	API errors
Interceptors	Request customization

Sample APIs

Method	Endpoint	Purpose
GET	/products	Fetch products
GET	/categories	Fetch categories
POST	/login	User login
POST	/register	User signup
POST	/cart	Add to cart

17. Authentication System

Authentication Workflow

Login Form



API Request



JWT Token



Local Storage



Protected Access

Authentication Features

Feature	Description
Login	User login
Signup	User registration
Logout	Remove token
Token Storage	Local storage

18. Cart Management System

Cart Features

Feature	Description
Add Product	Add items
Remove Product	Remove items
Increase Quantity	Quantity increment
Decrease Quantity	Quantity decrement
Total Calculation	Price calculation

Cart Workflow

Product Page



Add To Cart



Redux Updated



Cart Drawer Updated

19. Search Functionality

Search Features

Feature	Description
Real-Time Search	Instant search
Debouncing	Performance optimization
Suggestions	Product recommendations
Filtering	Category filtering

Search Workflow

User Input



Debounce Function



API Search



Display Results

20. Responsive Design

Supported Devices

Device Support

Mobile Full

Tablet Full

Laptop Full

Desktop Full

Tailwind Breakpoints

Breakpoint Size

sm 640px

md 768px

lg 1024px

xl 1280px

21. Performance Optimization

Optimization Benefit

Lazy Loading Faster loading

Memoization Prevent rerenders

Debouncing Efficient searching

Code Splitting Smaller bundle size

22. Animations & User Experience

Framer Motion Usage

Animation	Purpose
Banner Slider	Smooth transition
Cart Drawer	Slide animation
Hover Effects	Product interaction
Page Transition	Smooth routing

UX Features

Feature	Benefit
Skeleton Loader	Better loading experience
Sticky Navbar	Easy navigation
Toast Messages	User feedback
Infinite Scroll	Better browsing

23. Error Handling

Error Type	Solution
API Error	Toast message
Network Error	Retry mechanism
Invalid Route	404 page
Empty Cart	Empty cart UI

24. Testing Strategy

Testing Type	Tool
Unit Testing	Vitest

Testing Type	Tool
Component Testing	React Testing Library
E2E Testing	Playwright

Commands

```
npm run test
npm run build
npm run test:e2e
```

25. Git & GitHub Workflow

Git Workflow

Feature Branch



Commit Changes



Push to GitHub



Create Pull Request

Branch Naming

Type	Example
Feature	feature/navbar-ui
Bug Fix	fix/cart-issue
Docs	docs/readme-update

26. Build & Deployment

Build Command

npm run build

Deployment Platforms

Platform Purpose

[Vercel](#) Frontend hosting

[Netlify](#) Static deployment

Deployment Workflow

Development



Build



Deploy



Production

27. Future Enhancements

Feature	Benefit
Dark Mode	Better UI
Voice Search	Faster searching
AI Recommendations	Personalized shopping
PWA Support	App-like experience
Payment Integration	Complete checkout

28. Project Workflow

User Opens Website



Browse Products



Search Products



Add To Cart



Checkout



Order Success

29. Resume Description

Developed a Zepto-inspired Quick Commerce Frontend Application using ReactJS, TypeScript, Tailwind CSS, Redux Toolkit, React Router, Axios, and Framer Motion. Implemented responsive UI, reusable components, cart functionality, search system, and scalable frontend architecture.

30. Conclusion

TezKart is a modern quick-commerce frontend application developed using industry-standard frontend technologies and scalable architecture.

The project demonstrates:

- Modern React development
- Responsive frontend design
- State management

- API integration
- Component-based architecture
- Professional UI/UX practices

The application provides a strong foundation for real-world frontend development and scalable e-commerce solutions.

TezKart – Requirement Documentation

1. Functional Requirements

Functional requirements define what the system should do.

- User can register a new account
- User can log in securely
- User can browse all products
- User can view products category-wise
- User can search products instantly
- User can view product details
- User can add products to cart
- User can remove products from cart
- User can increase or decrease product quantity
- System should calculate total cart price automatically
- User can proceed to checkout
- User can filter products by category
- Navigation should work on all devices
- System should show success and error notifications
- Only logged-in users can access protected pages

2. Non-Functional Requirements

Non-functional requirements define how the system should behave.

- Website should be fully responsive for mobile, tablet, and desktop
- Pages should load quickly for better user experience
- Application should support scalable architecture
- Components should be reusable across the application
- Code should be clean, maintainable, and modular
- UI should support accessibility standards
- Application should work on all major browsers (Chrome, Firefox, Edge)
- Authentication should be secure using token-based system
- UI animations should be smooth and user-friendly
- System should handle errors properly with messages
- Performance should be optimized using lazy loading
- TypeScript should be used for type safety

3. User Requirements

User requirements define what users expect from the system.

- User wants easy and simple navigation
- User wants fast product search experience
- User wants smooth shopping experience
- User wants mobile-friendly interface
- User wants complete product information
- User wants simple checkout process
- User wants quick access to cart
- User wants instant feedback for actions

- User wants category-based browsing
 - User wants secure login system
 - User expects fast loading pages
 - User wants clear error messages
-

4. System Requirements

System requirements define the environment needed to run the application.

Software Requirements

- Node.js (version 20 or above)
 - npm (latest version)
 - React.js framework
 - TypeScript
 - Tailwind CSS for styling
 - Git for version control
 - VS Code for development
 - Modern web browser (Chrome, Firefox, Edge)
-

Final Output Style (Simple Summary)

- User can search products
- User can add products to cart
- User can remove products from cart
- Website should be mobile responsive
- System should be fast and scalable
- UI should be smooth and user-friendly